

Large Language Models - Esercizi Completi

Politecnico di Torino

27 gennaio 2026

Set Completo di Esercizi

Tokenization

Esercizio 1 (BPE Tokenization 1)

You are given the following corpus of text:

abababab

Apply Byte-Pair Encoding (BPE) to extract **3 tokens** (excluding 'a' and 'b'). Write the tokens and their extraction frequency (i.e., the frequency computed at the step when they were merged).

Note: when doing substitution, work from left to right.

Esercizio 2 (BPE Tokenization 2)

You are given the following sequence of characters:

eeeeee bcabbbb

Initially, each character is a separate token. Apply Byte-Pair Encoding to produce **3 additional tokens**. What are the generated tokens?

Write each token in the correct order. Separate the merged tokens with a comma (and no spaces). *Examples: 1) if merged tokens are aa and b, write: aa,b. 2) if merged tokens are x and y, write: x,y.*

Esercizio 3 (BPE Tokenization 3)

You are given the following sequence of characters:

aceaceaaaaacb

Initially, each character is a separate token. Apply Byte-Pair Encoding to produce **3 additional tokens**. What are the generated tokens?

Write each token in the correct order. Separate the merged tokens with a comma (and no spaces).

Esercizio 4 (BPE Tokenization 4)

You are given the following sequence of characters:

aaaabcbdddcdd

Initially, each character is a separate token. Apply Byte-Pair Encoding to produce **3 additional tokens**. What are the generated tokens?

Write each token in the correct order. Separate the merged tokens with a comma (and no spaces).

Attention Mechanism

Esercizio 5 (Attention Calculation 1)

The following vectors for the input tokens are given:

Inputs: $[2, 0]$, $[0, 2]$

The following matrices W_q , W_k , W_v are given:

$$W_q = \begin{bmatrix} 0 & 1 & 2 \\ 0 & 0 & 2 \end{bmatrix}, \quad W_k = \begin{bmatrix} 1 & 2 & 2 \\ 1 & 0 & 1 \end{bmatrix}, \quad W_v = \begin{bmatrix} 1 & 2 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

Compute Attention for the **first token**:

$$Attention(Q, K, V) = softmax\left(\frac{QK^T}{\sqrt{d_k}}\right) V$$

Esercizio 6 (Attention Calculation 2)

The attention for matrices Q, K, V (queries, keys, values) is computed as follows:

$$Attention(Q, K, V) = softmax\left(\frac{QK^T}{\sqrt{d_k}}\right) V$$

Where d_k is the dimensionality of the keys.

The following are the input tokens that reach an attention layer:

$$x = \begin{bmatrix} 3 & 4 & 1 \\ 0 & 4 & 0 \end{bmatrix}$$

The following are the W_q , W_k , W_v matrices for the attention layer:

$$W_q = \begin{bmatrix} -2 & -1 \\ 1 & 1 \\ 0 & -1 \end{bmatrix}, \quad W_k = \begin{bmatrix} 0 & -2 \\ -1 & 0 \\ -1 & 1 \end{bmatrix}, \quad W_v = \begin{bmatrix} 0 & 1 & -2 \\ 1 & 0 & 0 \\ -1 & -1 & -1 \end{bmatrix}$$

Compute the output of the attention operation for the **first token**. Report at least 4 significant figures.

Esercizio 7 (Attention Calculation 3)

The attention for matrices Q, K, V (queries, keys, values) is computed as follows:

$$Attention(Q, K, V) = softmax\left(\frac{QK^T}{\sqrt{d_k}}\right) V$$

Where d_k is the dimensionality of the keys.

The following are the two input tokens that reach an attention layer:

$$x = \begin{bmatrix} 0 & 0 & 0 \\ 4 & 0 & 0 \end{bmatrix}$$

The following are the W_q, W_k, W_v matrices for the attention layer:

$$W_q = \begin{bmatrix} -2 & -1 \\ 1 & 1 \\ 0 & -1 \end{bmatrix}, \quad W_k = \begin{bmatrix} 0 & -2 \\ -1 & 0 \\ -1 & 1 \end{bmatrix}, \quad W_v = \begin{bmatrix} 0 & 1 & -2 \\ 1 & 0 & 0 \\ -1 & -1 & -1 \end{bmatrix}$$

Compute the output of the attention operation for the **first token**. Report at least 4 significant figures.

Esercizio 8 (Attention Concepts 1)

What is the main difference between encoder-decoder cross-attention and encoder self-attention?

- In encoder-decoder cross-attention, the decoder attends only to the current token in the input sequence, while in encoder self-attention, the encoder attends to all tokens in the input.
 - Encoder self-attention attends to the encoder's output, while encoder-decoder cross-attention attends to the decoder's previous tokens.
 - In encoder self-attention, the decoder attends to the input sequence, whereas in encoder-decoder cross-attention, the encoder attends to the output sequence.
 - Encoder self-attention is used in the decoder, and encoder-decoder cross-attention is used in the encoder.
 - None of the other options is correct.
 - Encoder-decoder cross-attention attends only to the previous tokens in the output sequence, while encoder self-attention attends only to tokens in the input sequence.
 - Encoder self-attention only uses information from the input sequence, while encoder-decoder cross-attention allows the decoder to attend to the encoder's output.
-

Esercizio 9 (Attention Concepts 2)

In Causal Language Modeling, the word "causal" means that:

- The output is generated considering only the current token.
 - The output is generated considering only the previous tokens.
 - The output is generated considering only the previous and current tokens.
 - Only the decoder is used during inference.
 - The attention mechanism is adopted.
 - The surrounding context must also be used for token generation.
-

Parameters

Esercizio 10 (Parameter Counting 1 (Decoder-Only))

For the following **decoder-only** transformer model, compute:

1. The number of parameters for the embedding layer.
2. The number of parameters of the transformer (excluding the embedding layer, or head layer(s)).

Note: assume that none of the layers uses a bias term.

Hyperparameters:

- Vocabulary size $V = 30,000$
 - Embedding/model dimension $d = 768$
 - Positional embeddings: learned, for 512 positions
 - Number of layers $L = 12$
 - Number of attention heads $H = 12$
 - FF-NN projection dimension: 3072
 - Head dimension $d_k = d_v = 64$
-

Esercizio 11 (Parameter Counting 2 (Encoder-Only))

For the following **encoder-only** transformer model, compute:

1. The number of parameters for the embedding layer (token + positional).
2. The number of parameters of the transformer (excluding the embedding layer, or head layer(s)).

Note: assume that none of the layers uses a bias term. You can ignore layer normalization parameters.

Hyperparameters:

- Vocabulary size $V = 50,265$
- Embedding/model dimension $d = 1024$
- Positional embeddings: absolute, learned, for 514 positions
- Number of layers $L = 24$
- Number of attention heads $H = 16$
- FF-NN projection dimension: 4096
- Head dimension $d_k = d_v = 64$

Esercizio 12 (*Parameter Counting 3 (Encoder-Only)*)

You are given an **encoder-only** model. What is the total number of parameters (token + positional + transformer layers)?

Note: assume that none of the layers uses a bias term. You can ignore layer normalization parameters.

Hyperparameters:

- Vocabulary size $V = 128,000$
- Embedding/model dimension $d = 768$
- Positional embeddings: absolute, learned, for 2048 positions
- Number of layers $L = 36$
- Number of attention heads $H = 12$
- FF-NN projection dimension: 3072
- Head dimension $d_k = d_v = 64$

Report the result to the closest million (M).

- a. 185 M
- b. 255 M
- c. 100 M
- d. 52 M
- e. None of the other answers is correct
- f. 110 M
- g. 355 M
- h. 302 M
- i. 333 M

Esercizio 13 (*Pre-training Tasks*)

Which of the following tasks were used for the pre-training of BERT? Select all correct answers.

- a. Machine Translation
- b. Image-Text Matching
- c. Denoising Autoencoding
- d. Permuted Language Modeling
- e. Text Summarization
- f. Causal Language Modeling

- g. Next Sentence Prediction
 - h. Masked Language Modeling
 - i. Sentence Ordering Prediction
-

Coding & Concepts

Esercizio 14 (Code Analysis)

What is the likely output of the following code snippet?

```
1 from transformers import AutoModel, AutoTokenizer
2
3 model_name = "bert-base-uncased"
4 model = AutoModel.from_pretrained(model_name)
5 tokenizer = AutoTokenizer.from_pretrained(model_name)
6
7 prompt = "The result of the operation 2+2 is "
8 tokens = tokenizer(prompt, return_tensors="pt")
9
10 result = model.generate(**tokens)
11 print(tokenizer.batch_decode(result))
12
```

Provide a brief explanation (max 50 words). If an error occurs, indicate the line number and briefly explain why.

Esercizio 15 (Model Architectures)

Which one of these models is a decoder-only?

- a) BERT
 - b) GPT-2
 - c) T5
 - d) All options are decoder-only models
-

Esercizio 16 (Sampling)

Which one of these is a deterministic sampling approach? Select all correct answers.

- a) Temperature sampling with $T = 0$
 - b) Temperature sampling, with $0 < T < 1$
 - c) Temperature sampling, with $T > 1$
 - d) Beam Search
 - e) Greedy sampling
 - f) Top-k sampling
 - g) Top-p sampling
-

In-Context Learning & Instruction Tuning

Esercizio 17 (*In-Context Learning Novelty*)

What is the main novelty of In-Context Learning?

- a. It encodes the task in a prompt using a separate encoder network.
 - b. It allows achieving better performance than fine-tuning based approaches.
 - c. It improves generalization by updating the model's internal memory.
 - d. It allows to perform new tasks without parameter updates.
 - e. It trains a smaller model to mimic a larger teacher model.
 - f. It stores the examples in a retriever-based memory module.
 - g. It fine-tunes the model weights based on the provided examples.
 - h. None of the other answers is correct.
-

Esercizio 18 (*In-Context Learning Paradigm*)

In-context Learning has been a fundamental paradigm shift for Language Models because:

- a. It replaces the need for attention mechanisms by introducing a sequential input processing.
 - b. It improves the efficiency of training by compressing longer sequences into fixed-size representations.
 - c. None of the other options is correct.
 - d. It focuses on training models to explicitly output the reasoning steps required for solving a task, instead of directly giving the answer.
 - e. It introduces the ability to train models with fewer data points by leveraging pre-trained embeddings.
 - f. It allows models to perform tasks by conditioning on examples provided in the input, without requiring task-specific fine-tuning.
 - g. It enables models to store task-specific knowledge directly in their architecture, removing the need for example inputs.
-

Esercizio 19 (*Instruction Tuning Goal*)

The main goal of instruction tuning is:

- a. To fine-tune language models to follow explicit instructions.
- b. To reduce the size of the model without sacrificing performance.
- c. To focus on improving the model's understanding of rare or unseen tokens.

- d. None of the others is the main goal of instruction tuning.
 - e. To increase the model's tokenization speed by optimizing the vocabulary size.
 - f. To train language models to independently learn new languages from scratch.
 - g. To enhance the model's ability to generate longer sequences without truncation.
-

Esercizio 20 (RLHF Reward)

The following reward can be used when fine-tuning a model on human feedback:

$$R(x, y) = r_{\theta}(x, y) - \beta \log \frac{\pi_{new}(y|x)}{\pi_{old}(y|x)}$$

Where r_{θ} is the output of a Reward Model, π_{new} and π_{old} are the new and the original policies, respectively (you can see them as the probabilities assigned by the new and original models).

What is the purpose of the term $\log \frac{\pi_{new}(y|x)}{\pi_{old}(y|x)}$?

- a. It increases the entropy of the output (i.e., it helps the model produce more "creative" answers).
 - b. It improves the model's capability of following instructions.
 - c. It prevents the model from collapsing into a model that always predicts a single token.
 - d. It helps make the model more deterministic.
 - e. None of the other answers is correct.
 - f. It prevents the new model from producing outputs that are too dissimilar from the original one.
 - g. It rewards the model when it generates an answer that humans will "like".
-

Metrics

Esercizio 21 (BERTScore)

Which of the following best describes how BERTScore evaluates text similarity?

- a. It measures sequence similarity using n-gram co-occurrence weighted by IDF scores.
- b. It uses a fine-tuned BERT model to classify whether the reference and candidate texts are semantically related or not.
- c. It predicts a similarity score by treating the evaluation task as a masked language modeling problem, where BERT predicts the score as the next token.
- d. It assigns similarity scores based on syntactic parse trees rather than token embeddings.
- e. It uses cosine similarity on static word embeddings trained with Word2Vec or GloVe.
- f. It applies a bag-of-words model to compare token frequency distributions between reference and candidate texts.

- g. It computes token-level similarity by aligning contextualized embeddings from a pre-trained transformer model and aggregating scores.
-

Esercizio 22 (BLEU Score Calculation)

- A model generates the sentence: **"the fast brown fox jumped the bored dog"**
- Whereas, the target sentence is: **"the quick brown fox jumps over the lazy dog"**

Compute the **BLEU-2** score for this generation. Remember:

$$BLEU-n = BP \cdot \exp \left(\frac{1}{n} \sum_i \log(\text{precision}_i) \right)$$

Brevity Penalty:

$$BP = \begin{cases} 1 & \text{if } g > r \\ e^{(1-\frac{r}{g})} & \text{if } g \leq r \end{cases}$$

Where precision_i is the precision for the specific i -gram, and g and r are the lengths of the generated and reference texts, respectively. Consider each word to be a token in this case.

Fine-Tuning & Quantization

Esercizio 23 (PEFT Techniques)

Which of the following is **NOT** a Parameter-Efficient Fine-Tuning (PEFT) technique?

- a) Dynamic quantization
 - b) Adapter layers
 - c) Prompt tuning
 - d) LoRA
-

Esercizio 24 (Adapter Layers)

Adapter layers are introduced to reduce the number of tunable parameters. They consist in:

- a. A task-specific head added to the model, which is fine-tuned while the rest of the model is kept frozen.
- b. A method for splitting a large model into smaller sub-models that are trained independently and later combined.
- c. Replacing the feedforward layers in Transformers with low-rank matrix decompositions to reduce memory usage.
- d. Small, trainable layers inserted into a pre-trained model, which are fine-tuned while the rest of the model is frozen.

- e. A technique to merge multiple pre-trained models into a single model with fewer parameters.
 - f. Additional layers that replace the attention mechanism in the Transformer architecture to simplify computation.
 - g. None of the others is a valid description of adapters.
-

Esercizio 25 (*LoRA Assumptions*)

In LoRA, what does the process assume about the weight update matrix?

- a) That it is low rank or is near low rank.
 - b) That it replaces the initial weight matrix.
 - c) That it is initialized with numbers taken from a normal distribution $\mathcal{N}(0, 1)$.
 - d) That it is fixed (or frozen) throughout training.
-

Esercizio 26 (*Quantization Difference*)

What is the primary difference between static and dynamic quantization?

- a) Dynamic quantization computes scaling values for each activation during inference.
 - b) Dynamic quantization applies to weights only, while static applies to weights and activations.
 - c) Static quantization is slower but less accurate than dynamic quantization.
 - d) Dynamic Quantization requires a calibration step before the quantization.
 - e) None of the others.
-

Esercizio 27 (*Quantization 1*)

Apply **absmax quantization** to the following values x_1, x_2, x_3, x_4 , to map them to int8.

$$x_1 = -5, \quad x_2 = 4, \quad x_3 = -1, \quad x_4 = 0$$

Write the quantized versions.

Esercizio 28 (*Quantization 2*)

Apply **absmax quantization** to the following values x_1, x_2, x_3, x_4 , to map them to int8.

$$x_1 = -7, \quad x_2 = 3, \quad x_3 = -2, \quad x_4 = 0$$

Write the quantized versions.
